Cómo funciona BE

Guía central para entenderlo, dirigirlo y defenderlo

Esta guía explica BE como sistema: qué problema intenta resolver, con qué piezas, con qué reglas, cómo viaja un dato y cómo se cambia sin perder el control. Va de lo macro a lo micro, en cinco niveles, con los nombres reales del código.

Es para dos lectores. Para Dirección, que dirige el proyecto y tiene que poder explicarlo sin depender de una IA. Y para un profesor o un profesional que quiera entender cómo funciona.

No reemplaza los entregables académicos ni el legajo: el legajo (`docs/legajo`) sigue siendo la fuente de verdad de lo que BE debe hacer. Esta guía describe lo que el código hace en el commit de referencia.

Commit de referencia: `e4512cb` (`main`, 2026-10-26), con la tanda del 2026-10-25 integrada. La APK `0.18.0-candidata.1` sale de `b64d0c4`: el `main` del 25/10 con su número de versión.

Cómo se arma: fuente Typst y láminas generadas desde código (`docs/guias/ENTENDER-BE`). Los diagramas son vectoriales y exactos: ninguno es una imagen generativa.

Versión del documento: 1.0 (borrador del 2026-10-28). Solo datos sintéticos.

Proyecto: BE, plataforma integrada de inteligencia en salud · trabajo final de Analista de Sistemas · Elián Bufi.

CONTENIDO

¿Qué hay en esta guía y cómo se lee? 2

Nivel 1 · El sentido del producto

¿Qué problema intenta resolver BE? 3
¿Qué trabajo facilita, y qué necesita el profesional para decidir? 4
¿Qué valor está demostrado, y qué sigue siendo una hipótesis? 5

Nivel 2 · Las piezas y cómo se comunican

¿Cuáles son las piezas de BE y cómo se hablan? 6
¿Qué corre en el teléfono, qué en el navegador y qué en el servidor? 7
¿Cómo es un pedido por dentro? 8
¿Dónde está cada cosa, y qué servicios de afuera usa BE? 9

Nivel 3 · Las reglas y dónde se hacen cumplir

¿Quién revisa un pedido, y en qué orden? 10
¿Cómo decide BE si un profesional puede ver o hacer algo? 11
¿Dónde vive cada regla: en el código o en la base? 12

Nivel 4 · Dos recorridos completos

Recorrido 1: ¿qué pasa cuando una persona registra una comida? 13
¿Qué queda guardado, y qué pasa si algo falla? 14
¿Cómo llega esa comida al escritorio del profesional? 15
¿Por qué «Analizar» calcula en cada lectura, en vez de guardar los totales? 16
Recorrido 2: ¿qué pasa cuando Administración suspende un área? 17
¿Qué cambia y qué se conserva al suspender? 18

Nivel 5 · Cambiar el sistema sin perder el control

¿Cómo cambia la base sin perder los datos que ya tiene? 19
¿Por qué la versión de un plan y la versión de la app son cosas distintas? 20
¿Cómo llega un cambio a test, y una APK al teléfono? 21
¿Qué demuestra cada tipo de prueba, y qué no? 22
¿Por qué «integrado», «desplegado» y «validado» no son lo mismo? 23

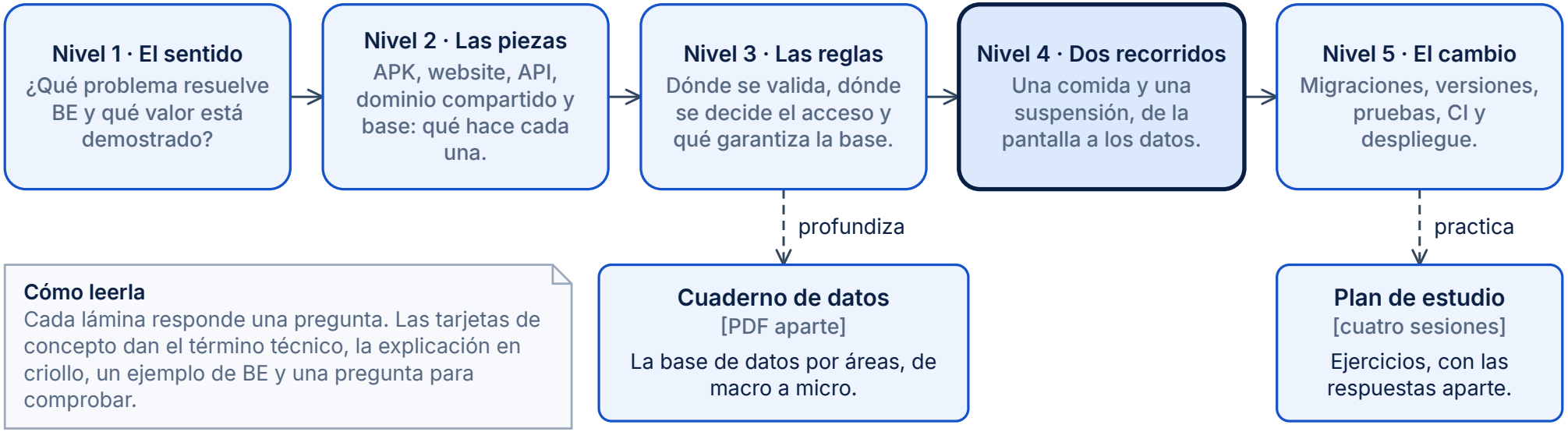
Para seguir

¿Por dónde seguir, y qué leer en el código? 24

BE · Guía central

¿Qué hay en esta guía y cómo se lee?

Cinco niveles, de lo macro a lo micro. Cada lámina responde una sola pregunta; la columna de la derecha, cuando la hay, trae los conceptos.



Los estados de una afirmación. Cada vez que la guía dice que algo existe, dice también en qué estado está. Los cinco primeros se prueban por separado y se combinan (nivel 5); «propuesto» y «fuera de alcance» no son un paso más: clasifican lo que no tiene código o no se va a hacer.

Implementado · Verificado · Integrado · Desplegado · Validado por personas

Propuesto · Fuera de alcance

Los colores de los módulos. Los mismos en las dos guías y en todos los diagramas por área:

Identidad, sesiones y Administración · Vínculos y consentimientos · Nutrición

Entrenamiento · Antropometría · Objetivos, procesos y revisiones

Auditoría, medios y plataforma

¿Qué problema intenta resolver BE?

La información de una persona que hace seguimiento con varios profesionales suele estar repartida, y nadie decide con la historia completa.

El problema. Una persona puede trabajar a la vez con una nutricionista, un entrenador y alguien que le toma medidas. Hoy, esa información suele vivir en planillas, PDF, fotos y chats. Cada profesional ve una parte, la persona no sabe bien quién mira qué, y cuando algo se corrige se pierde lo que había antes.

Lo que BE propone es un lugar común con tres reglas que el sistema hace cumplir, no solo promete:

1. **Cada profesional trabaja en su área sin mezclarse.** Nutrición, entrenamiento y antropometría son áreas separadas: un profesional opera solo las áreas en las que está verificado y habilitado.
2. **La persona decide quién accede, área por área.** Un vínculo por área, y una autorización que la persona da y puede revocar. Sin las dos cosas, el profesional no ve nada de esa persona.
3. **Lo registrado no se pisa.** Corregir agrega un registro nuevo; anular deja la historia. Un plan activado queda congelado como versión: para cambiarlo, se activa otra.

Quién usa qué:

Actor	Superficie	Para qué
Persona asesorada	APK de Android	Ver el plan del día, registrar comidas y entrenamientos, autorizar y revocar accesos.
Profesional	Website, `/pro`	Evaluar, fijar objetivos, armar y activar planes, analizar lo registrado y revisar.
Administración	Website, `/admin`	Verificar y habilitar profesionales por área, suspender y rehabilitar. Nunca ve datos de salud.

Lo que BE no hace, a propósito, y conviene decirlo antes de que lo pregunten:

- **No diagnostica ni califica.** Muestra lo registrado al lado de lo indicado; no dice «cumplió», «adhirió» ni da porcentajes (regla B05 del legajo, también en negación).
- **No inventa datos.** Lo que falta queda como faltante: un día sin registros no es un cero.
- **No certifica matrículas.** Habilitar para operar en BE no es certificar a un profesional.
- **No comparte nada sin autorización.** Ni siquiera Administración lee datos de salud.

CONCEPTO 1

Actor y rol

En criollo. Quién usa el sistema y para qué. Un actor es un tipo de usuario; su rol define qué puede hacer.
En BE. Tres actores: la persona asesorada (APK), el profesional (website, `/pro`) y Administración (website, `/admin`, con segundo factor). Una cuenta administrativa no puede ser profesional ni parte de un vínculo: lo impide la base.
Para comprobar. ¿Por qué conviene que la cuenta que suspende profesionales no pueda leer datos de salud?

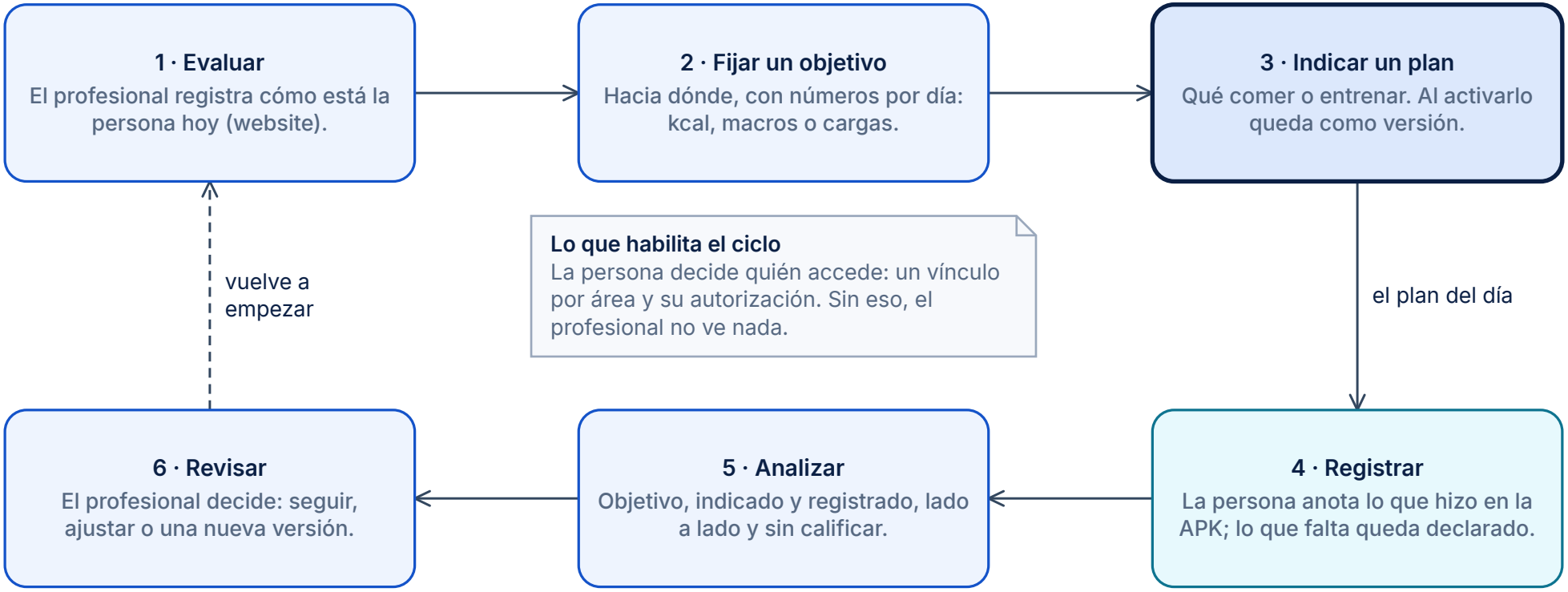
CONCEPTO 2

Dato sensible

En criollo. Un dato cuya exposición puede dañar a la persona. Los datos de salud lo son, y por eso se protegen más.
En BE. Lo que se registra de comidas, entrenamiento y medidas. La persona da un permiso general (A3) y, después, uno por cada profesional y área.
Para comprobar. ¿Qué cambia para el diseño si un dato es sensible?

¿Qué trabajo facilita, y qué necesita el profesional para decidir?

BE ordena un ciclo: evaluar, fijar un objetivo, indicar un plan, registrar, analizar y revisar. La persona habilita el ciclo.



CONCEPTO 3

Objetivo, indicado y registrado

En criollo. Tres cosas distintas: lo que fijó el profesional para el día, lo que el plan decía y lo que la persona anotó. Ninguna califica a las otras.
En BE. Prueba Dos: objetivo 1.800 kcal, indicado 1.572 kcal, y lo registrado de cada día.
Para comprobar. ¿Por qué un plan con alternativas no tiene «indicado del día»?

CONCEPTO 4

Versión

En criollo. Una foto fija de algo que cambia con el tiempo. Cambiar es crear otra versión, no pisar la anterior.
En BE. Al activar un plan, BE guarda una instantánea con la composición de cada alimento. Si mañana se corrige el catálogo, el plan de ayer sigue diciendo lo mismo.
Para comprobar. ¿Qué se pierde si un plan activado se pudiera editar?

CONCEPTO 5

Ausencia declarada

En criollo. Si falta un dato, se dice que falta. No se inventa ni se cuenta como cero.
En BE. Un día sin registros se ve en gris, no como 0 kcal. Un almuerzo sin cantidades hace del día un «subtotal de lo registrado».
Para comprobar. ¿Qué conclusión equivocada sacaría un profesional si un día sin registros se dibujara como cero?

¿Qué valor está demostrado, y qué sigue siendo una hipótesis?

Que el sistema funcione no prueba que sirva. Son preguntas distintas, con evidencias distintas.

Afirmación	Estado	Evidencia o lo que falta
BE hace cumplir sus reglas de acceso, historia y autorización	Verificado	909 pruebas de integración contra PostgreSQL real, recorridos en el navegador y la verificación de los escenarios de prueba (6 de 6, el 2026-10-26).
BE corre en un ambiente que se puede usar	Desplegado	Test sirve `e4512cb` en la API y el website. La APK `0.18.0-candidata.1` está publicada como prerelease; la estable sigue siendo la 0.13.2.
Administración se entiende sin ayuda	Validado en parte	Parte A de la prueba breve, hecha por Dirección el 2026-10-27: las siete filas coincidieron, con cuatro observaciones de comprensión.
Un profesional lee bien «Analizar» (tres capas)	Pendiente	Parte B de la prueba breve y la prueba con profesionales (DL-153, DL-154 y DL-158).
Una persona registra sin ayuda en el teléfono	Pendiente	Parte C y el guion completo del teléfono, con TalkBack.
BE ahorra tiempo o mejora decisiones	Hipótesis	No se midió. La auditoría del 2026-10-21 lo marca (VAL-01): los objetivos de la tesis son propiedades del sistema, no beneficios medidos (ACA-01).
La cartera de un profesional responde con 200 personas	Medido fuera de Render	En un runner de GitHub (28/10): 1,2 s con 200 personas, unos 6 ms por persona, en línea. No dice nada de Render ni de varios profesionales a la vez.

Para defenderlo frente a un profesor
Separar las dos cosas da fuerza, no la quita: «el sistema garantiza estas reglas, y lo demuestran estas pruebas; que le sirva a un profesional es la hipótesis que sigue, y así la vamos a validar».

CONCEPTO 6

Verificación y validación

En criollo. Verificar es comprobar que el sistema hace lo que se especificó («¿lo construimos bien?»). Validar es comprobar que eso le sirve a quien lo usa («¿construimos lo correcto?»).
En BE. Las 909 pruebas de integración verifican. La prueba breve con Dirección y la prueba con profesionales validan, y todavía no están completas.
Para comprobar. Una prueba automática pasa en verde. ¿Qué sabés y qué no sabés todavía?

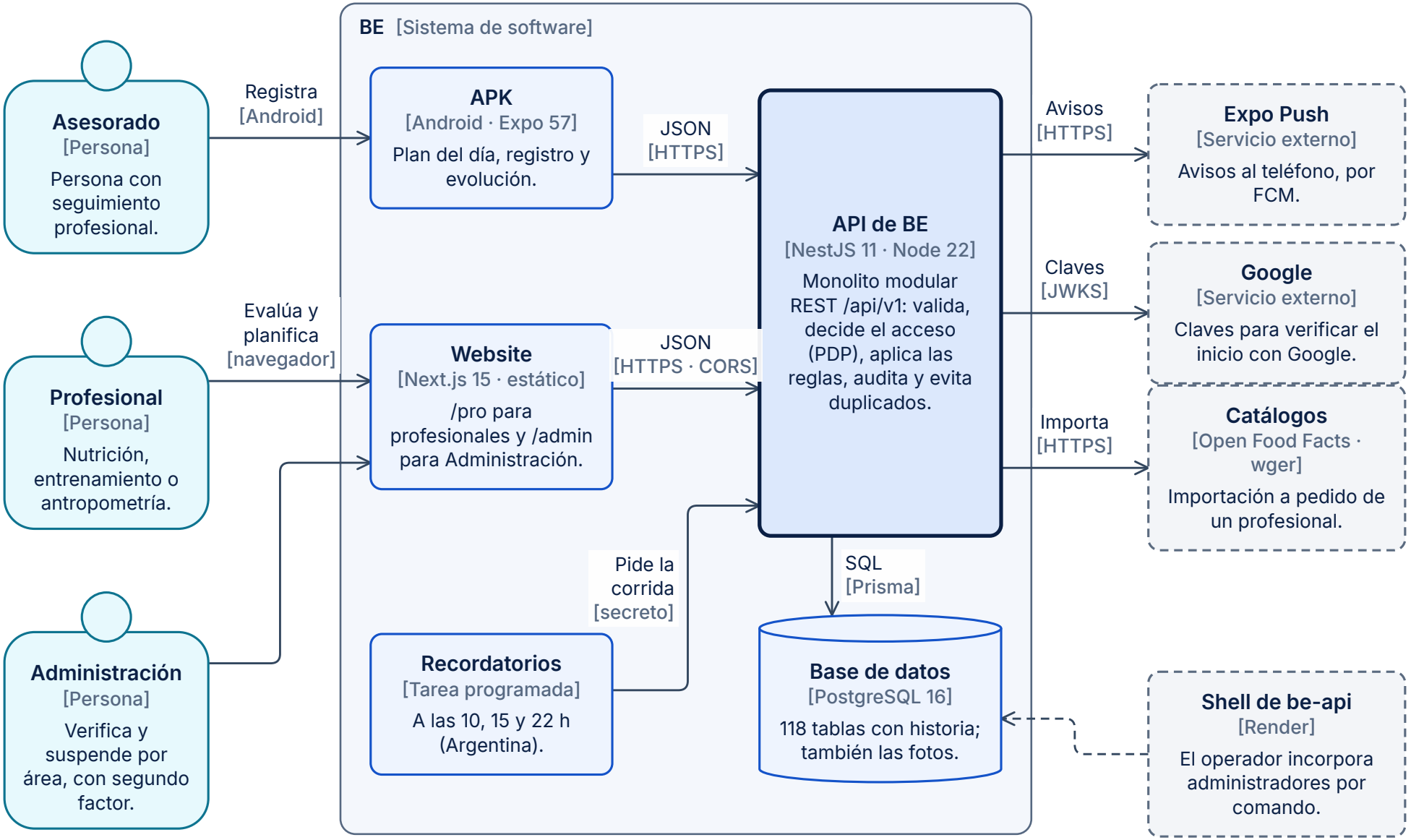
CONCEPTO 7

Hipótesis de valor

En criollo. Lo que creemos que el producto le aporta a alguien, antes de haberlo medido.
En BE. «Ver objetivo, indicado y registrado lado a lado ayuda a revisar mejor» es una hipótesis: nadie la midió con profesionales reales.
Para comprobar. ¿Cómo la pondrías a prueba con tres profesionales en una semana?

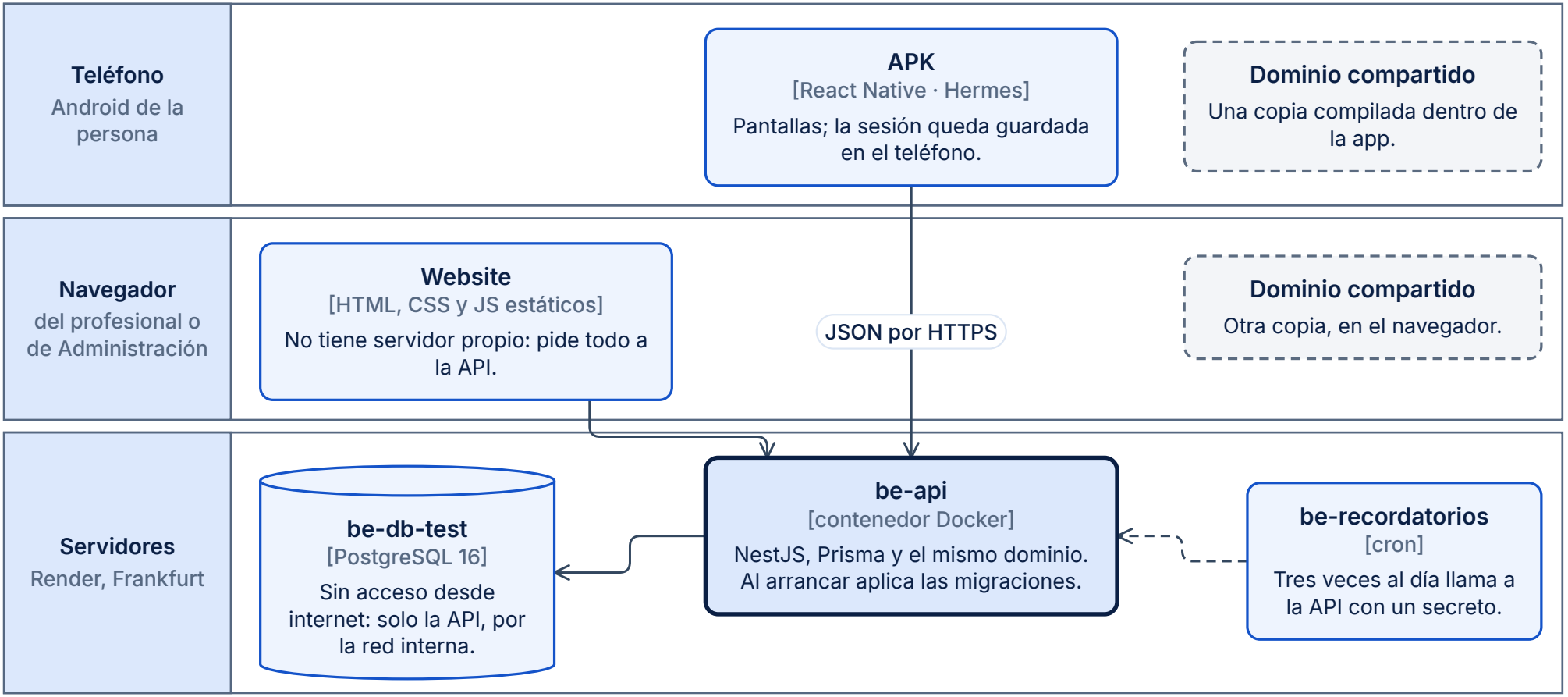
¿Cuáles son las piezas de BE y cómo se hablan?

Tres superficies para tres actores, una sola API que decide y una sola base por ambiente.



¿Qué corre en el teléfono, qué en el navegador y qué en el servidor?

Las pantallas corren en el aparato de cada uno; las decisiones, solo en el servidor.



CONCEPTO 8

Cliente y servidor

En criollo. El cliente muestra y pide; el servidor guarda y decide. Un cliente puede estar viejo o adulterado: el servidor no le cree.

En BE. La APK y el website son clientes. La API (`be-api`) es el servidor: cada regla que importa se comprueba ahí, aunque la pantalla ya la haya mirado.

Para comprobar. Si una APK con un error dejara mandar una comida con fecha de mañana, ¿qué pasaría con ese pedido?

CONCEPTO 9

Dominio compartido

En criollo. Las reglas y los contratos escritos una sola vez y usados en varias partes.

En BE. `packages/domain` se compila dentro de la API, del website y de la APK: los tres validan con los mismos esquemas y calculan con las mismas funciones (por ejemplo, `calcularNutrientes`).

Para comprobar. Si el dominio también corre en el teléfono, ¿por qué la API vuelve a validar todo?

¿Cómo es un pedido por dentro?

Registrar una comida es un pedido HTTP: método, ruta, encabezados y un cuerpo JSON. La respuesta trae un código y otro JSON.

EL PEDIDO QUE MANDA LA APK

```
POST /me/nutrition/meal-records HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.…
Idempotency-Key: 6f1c2a9e-almuerzo-intento-1
Content-Type: application/json

{
  "kind": "PLAN_OPTION",
  "activePlanId": "8d0e…",
  "dayTypeId": "c41e…",
  "mealId": "e09f…",
  "optionId": "7a2b…",
  "occurredAt": "2026-10-27T13:05:00-03:00",
  "consumption": { "status": "PLAN_PORTIONS" },
  "observation": null
}
```

La ruta elige la operación: `API-ING-02`. **Authorization** lleva el token de la sesión: un JWT firmado que solo dice qué fila de `sesion` es. **Idempotency-Key** nombra este intento: si la red se corta y la APK reintenta con la misma clave, recibe la misma respuesta y no se registra dos veces.

LA RESPUESTA SI SALE BIEN

```
HTTP/1.1 201 Created

{ "data": {
    "recordId": "5b7a…",
    "kind": "PLAN_OPTION",
    "localDate": "2026-10-27",
    "meal": { "mealId": "e09f…", "label": "Almuerzo" },
    "consumed": {
      "energyKcal": { "value": "640.5", … }, … },
    "annulment": null, … } }
```

Y SI EL PLAN YA NO ES EL VIGENTE

```
HTTP/1.1 422 Unprocessable Entity

{ "error": { "code": "ACTIVE_PLAN_REQUIRED",
    "message": "El plan que estás viendo ya no es el
vigente…" } }
```

Los números viajan como texto decimal exacto («640.5») y se redondean solo al mostrarlos. Todo error tiene la misma forma: un `code` para el programa y un `message` para la persona.

CONCEPTO 10

Contrato

En criollo. El acuerdo sobre qué se manda y qué se recibe en cada operación, escrito de forma que una máquina lo pueda comprobar.
En BE. Esquemas zod en `packages/domain`: `RegistrarComidaRequestSchema` rechaza un campo de más o un tipo equivocado.
Para comprobar. ¿Qué gana BE al rechazar un campo que no conoce, en vez de ignorarlo?

CONCEPTO 11

Código de estado HTTP

En criollo. Un número que resume cómo terminó el pedido: 2xx salió bien, 4xx el pedido no corresponde, 5xx falló el servidor.
En BE. 400 forma, 401 sesión, 403 área no operable, 404 sin permiso o inexistente, 409 conflicto, 422 regla.
Para comprobar. ¿Por qué no alcanza con responder 400 a todo lo que sale mal?

CONCEPTO 12

Idempotencia

En criollo. Repetir un pedido tiene el mismo efecto que hacerlo una vez.
En BE. `registro_de_idempotencia` guarda la clave, una huella del pedido y la respuesta. Misma clave y mismo pedido: la misma respuesta. Misma clave y otro pedido: 409.
Para comprobar. Un teléfono sin señal reintenta tres veces. ¿Cuántas comidas quedan registradas, y por qué?

¿Dónde está cada cosa, y qué servicios de afuera usa BE?

Un solo repositorio con todo el sistema. Los servicios externos ayudan, pero ninguno guarda datos de salud.

EL REPOSITORIO

```
be/
├─ apps/
│   ├─ api/      NestJS: una carpeta por área
│   │            sesion, identidad, vinculo,
│   │            consentimiento, autorizacion (PDP),
│   │            nutricion, entrenamiento,
│   │            antropometria, analisis,
│   │            administracion, plataforma, http
│   ├─ web/      Next.js exportado a archivos fijos
│   └─ mobile/   Expo y React Native: la APK
├─ packages/domain/   reglas, contratos, cálculos
├─ prisma/
│   ├─ schema.prisma   118 modelos, 84 enumeraciones
│   └─ migrations/     53, con los disparadores
├─ test/integration/  67 archivos, PostgreSQL real
├─ test/navegador/    los 17 recorridos
├─ docs/legajo/       la fuente de verdad
├─ EVIDENCIA/         qué se probó y cómo
├─ render.yaml        los servicios de test
└─ .github/workflows/ci.yml   la CI
```

Dentro de cada carpeta de la API, el `*.controller.ts` recibe el pedido (la ruta y sus guards); el `*.service.ts` aplica las reglas dentro de una transacción; los `lectura-*.ts` y `fuentes.ts` arman las consultas de lectura. Lo que no depende de la base (reglas puras, contratos, cálculos) vive en `packages/domain`.

LOS SERVICIOS DE AFUERA

Servicio	Para qué	Si no responde
Render	Aloja la API, el website, la base de test y el cron.	Test no responde. Los datos siguen en la base.
Expo Push y FCM	Llevar un aviso al teléfono, después de guardar.	El aviso no llega; la novedad se ve al abrir la app.
Google	Entrar con Google: BE comprueba la firma del token con las claves públicas de Google.	No se entra con Google; quien tiene contraseña, entra con ella.
Open Food Facts y wger	Importar un alimento o un ejercicio, a pedido del profesional.	No se importa. El catálogo base ya está en la base: 433 alimentos y 198 ejercicios.
GitHub	Código, CI y las APK publicadas.	No se integra ni se despliega; test sigue con lo anterior.

CONCEPTO 13

Monorepo

En criollo. Un solo repositorio para varias aplicaciones que comparten código.
En BE. Un cambio de contrato toca el dominio, la API y los clientes en el mismo PR, y la CI los prueba juntos.
Para comprobar. ¿Qué riesgo aparece cuando la API y la APK viven en repositorios separados?

¿Quién revisa un pedido, y en qué orden?

Varias capas, cada una con su pregunta. La pantalla ayuda; la que decide es la API, y la base es la última red.



CONCEPTO 14

Forma y regla de negocio

En criollo. La forma pregunta si el pedido está bien escrito; la regla, si tiene sentido ahora. Una fecha válida puede ser una fecha futura.
En BE. La forma la revisa zod (400) antes de tocar la base. La regla la revisa el servicio con datos de la base (422 o 409): «el plan que mandás ya no es el vigente».
Para comprobar. «Cantidad: −3 g»: ¿es un problema de forma o de regla? ¿Y «comida del 30/10» si hoy es 28/10?

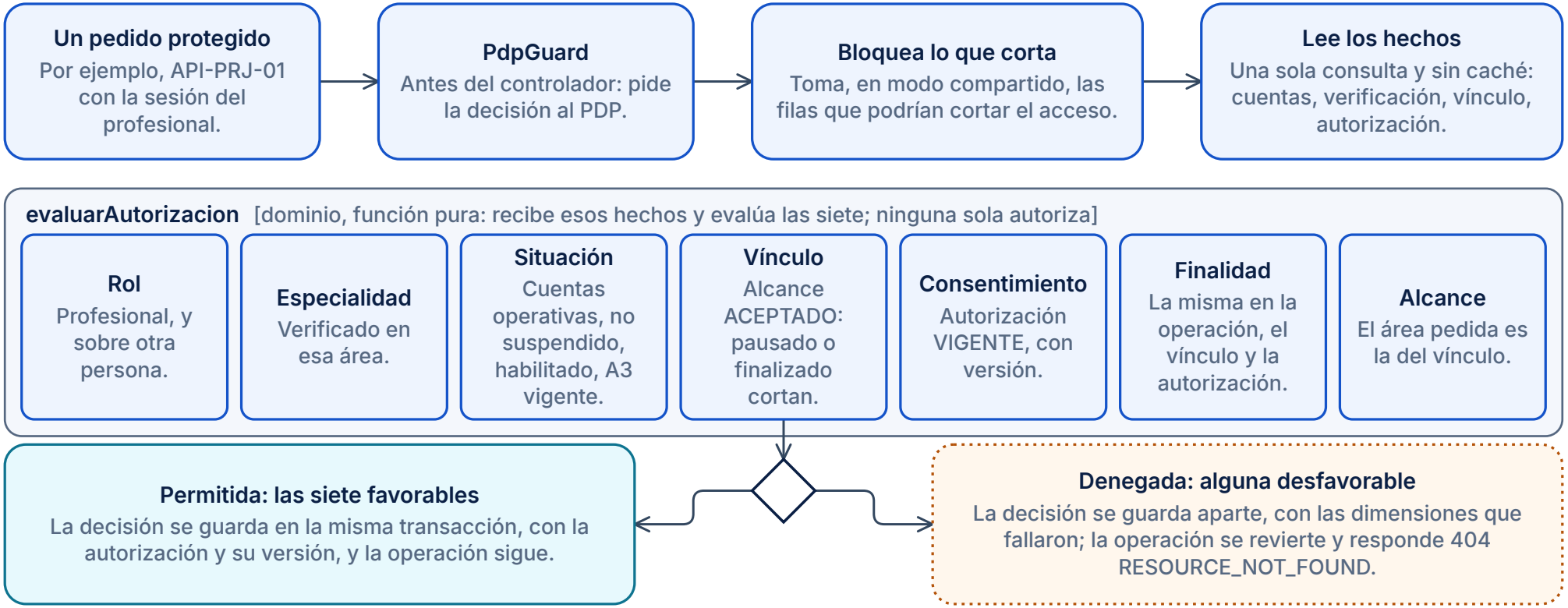
CONCEPTO 15

Defensa en profundidad

En criollo. Varias barreras independientes, para que el error de una no alcance para romper el sistema.
En BE. El servicio impide editar un plan activado; si un defecto lo dejara pasar, el disparador `version_de_plan_nutricional_guardar` lo rechaza igual y la transacción se revierte.
Para comprobar. ¿Por qué una barrera de más en la base no reemplaza una prueba del servicio?

¿Cómo decide BE si un profesional puede ver o hacer algo?

Un solo punto de decisión, siete preguntas; alcanza con una desfavorable para negar.



EXTRACTO DEL CÓDIGO

```
// packages/domain/src/autorizacion.ts
const situacionFavorable =
  h.actor.cuentaOperativa &&
  titular !== null &&
  titular.cuentaOperativa &&
  titular.a3Vigente &&
  h.verificacion !== 'SUSPENDIDO' &&
  h.habilitacion === 'CONCEDIDA';
if (!situacionFavorable)
  desfavorables.push('SITUACION');
```

CONCEPTO 16

Punto de decisión (PDP)

En criollo. El único lugar donde se decide si una operación está permitida. Las demás partes preguntan.
En BE. `PdpService` lee los hechos sin caché y llama a `evaluarAutorizacion`, una función pura que se prueba sin base.
Para comprobar. Un profesional suspendido sigue verificado en su especialidad. ¿Qué dimensión lo frena?

CONCEPTO 17

404 en lugar de 403

En criollo. Sin permiso sobre algo de otra persona, el sistema no te dice si existe.
En BE. Sin permiso sobre una persona: 404, como si no existiera. 403 queda para lo que no expone a nadie, como las plantillas de un área suspendida.
Para comprobar. ¿Qué le revelaría un 403 a quien prueba identificadores al azar?

¿Dónde vive cada regla: en el código o en la base?

Las reglas de negocio se dicen en el dominio y el servicio. Las que protegen la historia se repiten en la base, que no se puede saltear.

Regla	Dónde se hace cumplir	Qué responde
Una cuenta administrativa no es profesional ni parte de un vínculo	Base: `be_sin_rol_administrativo`, en seis tablas	Se revierte todo
Nadie decide sobre su propia verificación	Base: un disparador de `evento_de_verificacion`	Se revierte todo
Un plan activado no se modifica	Servicio y base: `version_de_plan_nutricional_guardar` exige una sucesora	422
Lo registrado solo se agrega	Base: `ingesta_nutricional_solo_agregar` y `be_solo_agregar` en las demás	Corregir es otra fila
Una comida efectiva por comida y día	Servicio, y el índice único parcial `ingesta_nutricional_prescripta_unica` para dos pedidos simultáneos	200 si es la misma; 409 si es otra
Suspender solo desde «verificado»	Dominio (`evaluarDecision`) y base (`be_verificacion_profesional_guardar`)	422
Qué puede ver un profesional	PDP: `evaluarAutorizacion`, con la decisión guardada	404
No calificar a la persona (B05)	Los textos de las pantallas y su prueba (`copy-pantallas.test.cjs`)	No es un dato

Ocultar un botón no es autorizar

El website usa `accessMode` y el estado del área para no ofrecer acciones que no corresponden: es una ayuda. La API vuelve a decidir en cada pedido. Quien llame a la API sin pasar por la pantalla recibe el mismo 404.

CONCEPTO 18

Disparador (trigger)

En criollo. Una función que la base ejecuta sola ante un cambio en una tabla, y que puede rechazarlo.
En BE. Hay 292 vigentes. Casi todos cuidan dos cosas: que la historia solo se agregue y que los estados cambien solo por transiciones declaradas («TRANSICION_NO_DECLARADA»).
Para comprobar. Si el servicio ya controla la regla, ¿para qué repetirla en la base?

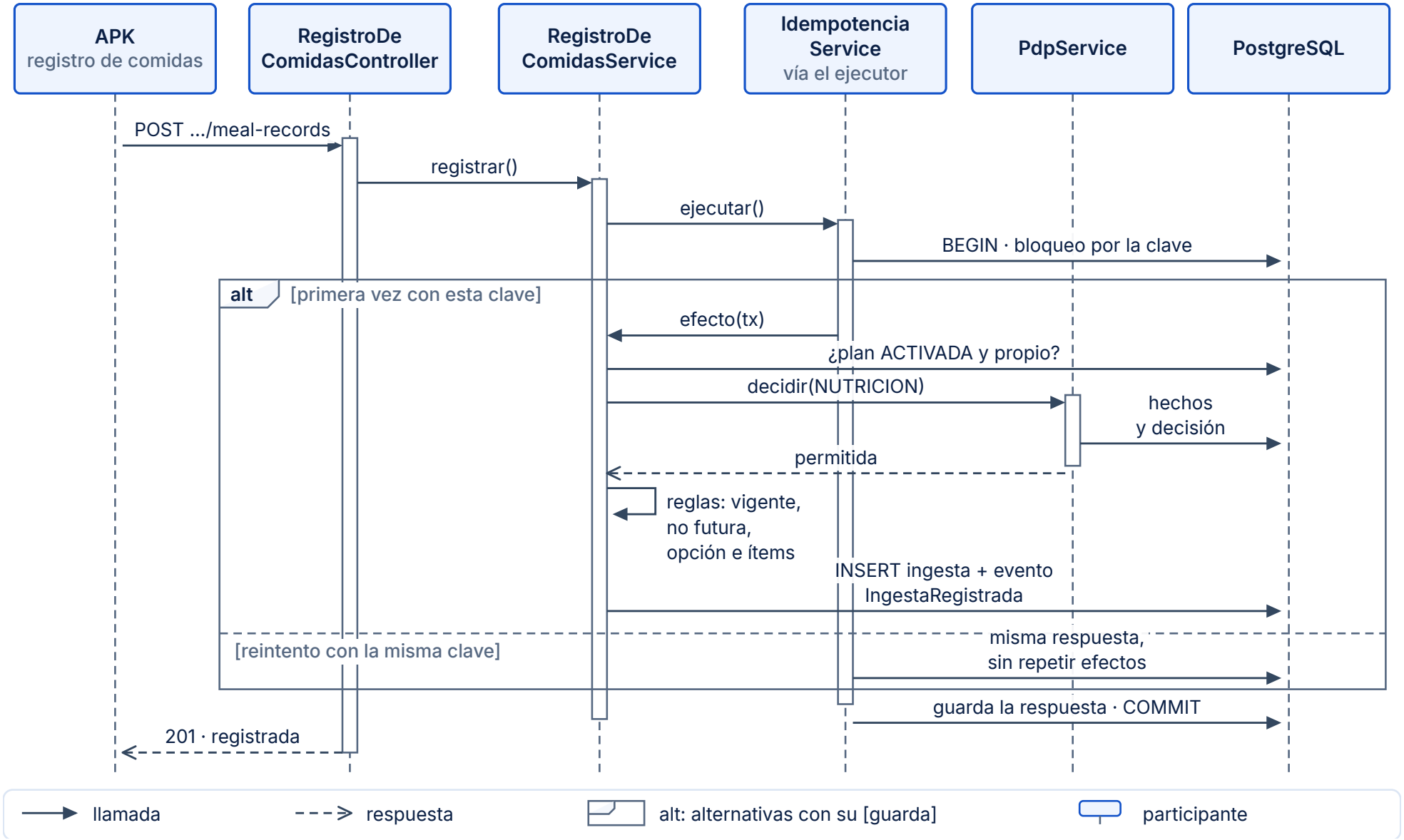
CONCEPTO 19

Sesión y permiso

En criollo. La sesión dice quién sos (autenticación). El permiso dice qué podés hacer, sobre quién y en qué área (autorización).
En BE. La sesión dura hasta 12 horas y se comprueba en cada pedido (401). El permiso se decide en cada pedido con el PDP (404). Revocar una autorización no cierra la sesión del profesional: le corta ese permiso.
Para comprobar. Una persona revoca la autorización mientras el profesional mira su ficha. ¿Qué ve el profesional en el pedido siguiente?

Recorrido 1: ¿qué pasa cuando una persona registra una comida?

En la APK, en «Hoy», la persona elige una opción del almuerzo y toca «Registrar comida».



¿Qué queda guardado, y qué pasa si algo falla?

Todo se escribe en una sola transacción: o queda la comida con su historia y su auditoría, o no queda nada.

Paso	En este recorrido
Pantalla	«Hoy», en la APK: la opción elegida, la hora y, si se quiere, las cantidades.
Operación	`POST /me/nutrition/meal-records` (`API-ING-02`), con la sesión y una clave por intento.
Reglas	Sesión; forma; plan activado y propio; permiso del PDP sobre Nutrición; plan vigente; comida no futura; la opción y los ítems son del plan; una por comida y día.
Datos	`ingesta_nutricional` (en una «comida diferente» con fotos, también `evidencia_visual_de_ingesta`), `evento_de_nutricion` «IngestaRegistrada», `decision_de_acceso`, `registro_de_auditoria` y `registro_de_idempotencia`.
Respuesta	201, con el registro tal como quedó y lo consumido calculado con la instantánea del plan.

Respuesta	Cuándo	Qué queda guardado
401	La sesión no existe, venció o fue revocada.	Nada.
400	El cuerpo no cumple el contrato, o falta la clave de idempotencia.	Nada.
404	El plan no es suyo, o el PDP la deniega (por ejemplo, el área del profesional está suspendida).	La decisión denegada y el rechazo en la auditoría, aparte.
422	El plan ya no es el vigente, la comida es futura o la opción no es del plan.	Nada de la comida; el rechazo queda en la auditoría.
409	Ya hay otra comida registrada en esa comida y ese día, o la clave se usó con otro pedido.	Nada nuevo; el rechazo queda en la auditoría.
200	Es exactamente la misma comida que ya estaba.	Nada nuevo: devuelve la que estaba.

CONCEPTO 20

Transacción

En criollo. Un grupo de cambios que se confirma entero o no se confirma: no hay mitades.
En BE. La comida, su evento, la decisión del PDP, la auditoría y la respuesta guardada se confirman con un solo `COMMIT`. Si una regla falla en el medio, se revierte todo.
Para comprobar. Si la auditoría se guardara en otra transacción, después de la comida, ¿qué podría quedar mal?

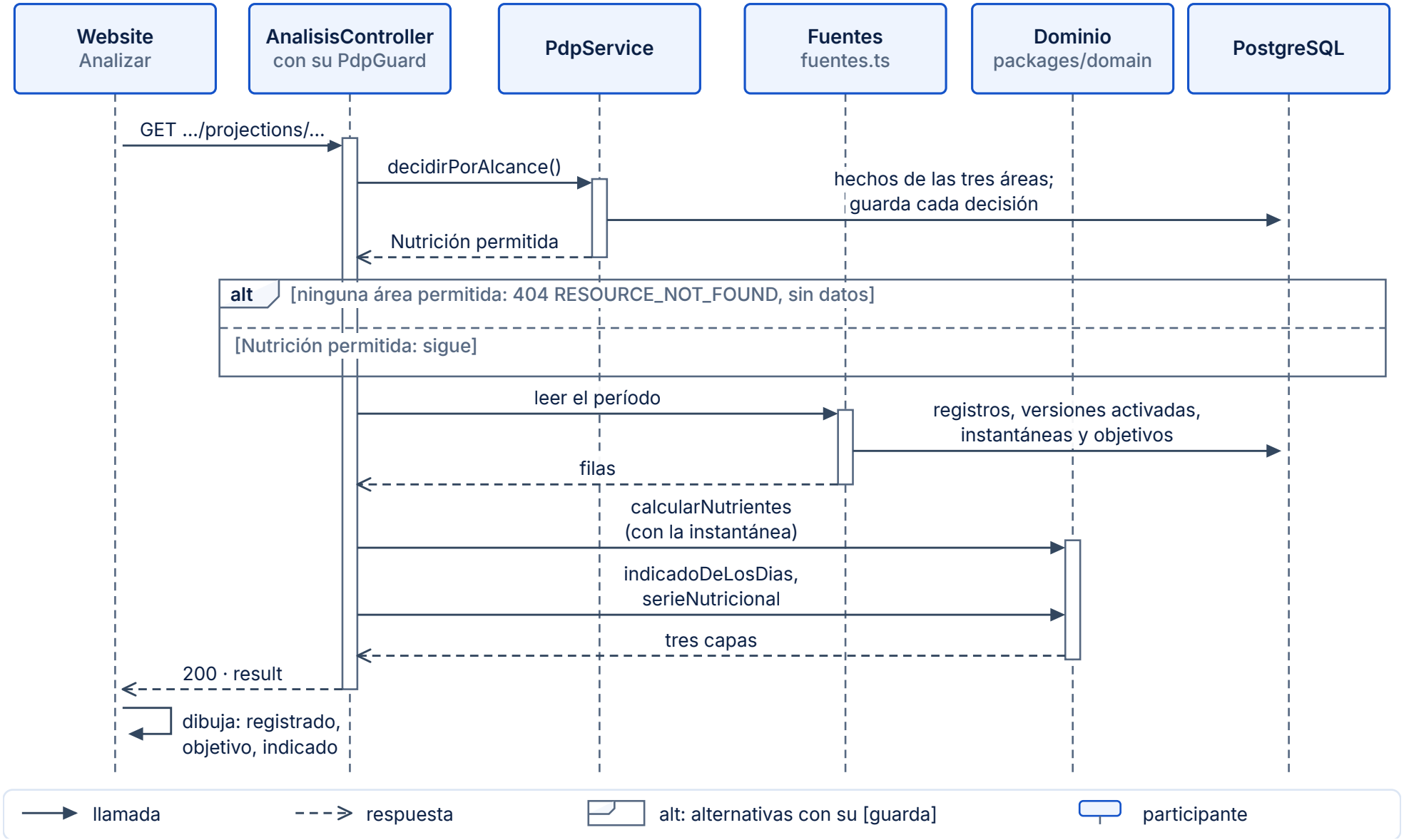
CONCEPTO 21

Auditoría

En criollo. Un registro de quién hizo qué, cuándo y con qué resultado, que no se puede reescribir.
En BE. `registro_de_auditoria` anota la operación, el actor, el resultado (éxito o rechazo) y el motivo, sin el contenido de salud.
Para comprobar. ¿Por qué la auditoría no guarda qué comió la persona?

¿Cómo llega esa comida al escritorio del profesional?

En «Analizar», el website pide una proyección. Nada se precalcula: cada lectura decide el permiso y calcula de nuevo.



¿Por qué «Analizar» calcula en cada lectura, en vez de guardar los totales?

Lo guardado son los hechos: qué se registró y contra qué versión del plan. Los totales se derivan, siempre con la misma regla.

Qué pide el website. `API-PRJ-01` con la persona y el período. Antes del controlador, el `PdpGuard` pide al PDP una decisión por cada área (`decidirPorAlcance`) y las guarda. Si no hay ninguna permitida, responde 404 sin datos. Si hay alguna, sigue, y las áreas negadas vuelven marcadas como no disponibles, sin sus datos.

Qué lee. `fuentes.ts` trae las comidas del período, las versiones activadas que las respaldan, sus instantáneas y los objetivos. No trae nada de otra persona ni de un área negada.

Qué calcula. En el dominio: `calcularNutrientes` suma lo consumido con la instantánea; `indicadoDeLosDias` saca lo que el plan indicaba para cada día, y `serieNutricional` arma las tres capas que se dibujan:

Capa	Qué es	De dónde sale
Registrado	Lo que la persona anotó.	Las comidas y la instantánea.
Objetivo	Lo que fijó el profesional para el día.	La versión del objetivo vigente.
Indicado	Lo que decía el plan para ese día.	La versión activada del plan. Si un día tiene alternativas, no hay un solo indicado.

Lo que no hace. No completa los días sin registros: los muestra como faltantes. No suma lo que no se puede calcular: si a una comida le faltan cantidades, el día dice «subtotal de lo registrado». No compara para calificar: pone las tres capas una al lado de la otra (B05).

Para leer en el código: `apps/api/src/analisis/analisis.controller.ts` · `apps/api/src/analisis/fuentes.ts` · `packages/domain/src/nutricion-del-analisis.ts`

CONCEPTO 22

Dato derivado

En criollo. Un dato que se calcula a partir de otros y que, por eso, no hace falta guardar.
En BE. Las kilocalorías del día salen de las comidas registradas y de la composición congelada en la instantánea: `SUM_SOURCE_PER_100G_V1`. Si se guardaran, habría que mantenerlas al día en cada corrección.
Para comprobar. Una persona rectifica las cantidades del almuerzo. ¿Qué hay que actualizar para que «Analizar» lo muestre bien?

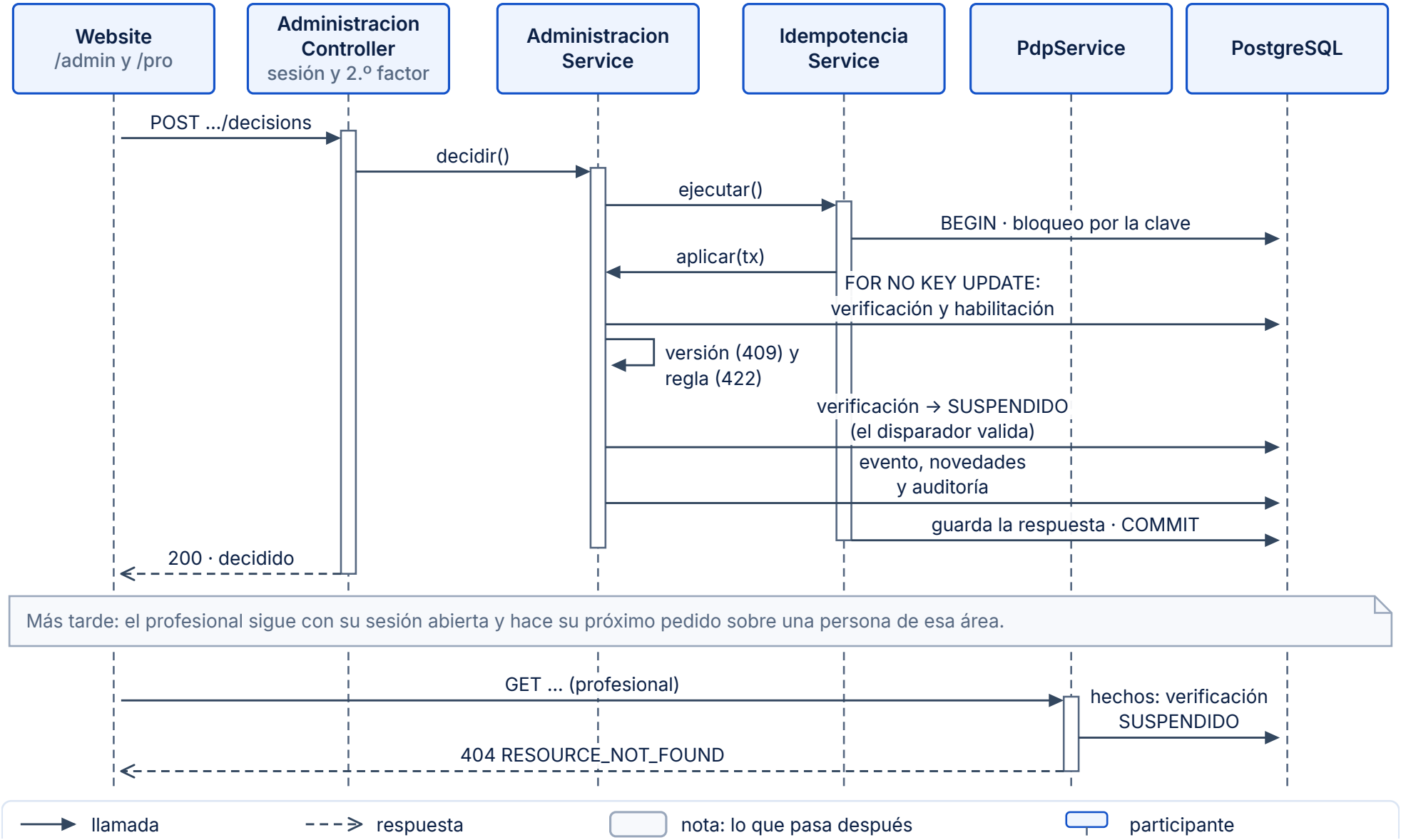
CONCEPTO 23

Instantánea

En criollo. Una copia fija de los datos de los que dependía una decisión, tomada en el momento en que se decidió.
En BE. Al activar un plan se guarda `instantanea_de_plan_nutricional`, con la composición de cada alimento y una huella SHA-256. Las comidas se calculan con ella, no con el catálogo de hoy.
Para comprobar. ¿Qué pasaría con los gráficos de octubre si se calcularan con el catálogo corregido en noviembre?

Recorrido 2: ¿qué pasa cuando Administración suspende un área?

En `/admin`, con segundo factor, Administración suspende Nutrición de un profesional, con un fundamento y un mensaje.



¿Qué cambia y qué se conserva al suspender?

Suspender cambia un estado y deja su historia. No borra nada y no toca lo que es de la persona.

Cambia

- La verificación de Nutrición pasa de VERIFICADO a SUSPENDIDO, con su evento, en una transacción.
- El profesional, con la misma sesión: lo de Nutrición de sus personas responde 404; las plantillas de Nutrición responden 403, y Entrenamiento sigue igual.
- La persona: ve su historia, «Hoy» dice que el plan no está disponible y no puede registrar en ese plan (404).
- Avisos: una novedad al profesional y a cada persona vinculada (también en pausa), sin el fundamento. El aviso al teléfono sale después del `COMMIT`.

Se conserva

- Planes, comidas, borradores y evaluaciones: no se borra nada. La prueba cuenta antes y después.
- Los vínculos y las autorizaciones de las personas, tal como estaban.
- La habilitación y las otras áreas del profesional.
- La sesión del profesional: no se lo echa; se le corta el área.

Al rehabilitar (`REINSTATE`), la verificación vuelve a VERIFICADO y todo vuelve a operar. Una predicción común es que haya que volver a vincular y autorizar: no hace falta, porque el vínculo y la autorización son de la persona y no se tocaron. Lo que la persona sí cortó durante la suspensión (revocar la autorización, finalizar el vínculo) sigue cortado: rehabilitar no lo devuelve.

Para leer en el código: `test/integration/administracion.int-spec.ts (suspender Nutrición corta esa área desde el pedido siguiente)` · `apps/api/src/administracion/administracion.service.ts`

CONCEPTO 24

Estado e historia

En criollo. El estado es cómo está algo ahora; la historia, cómo llegó hasta ahí. Un buen modelo guarda las dos.

En BE. `verificacion_profesional` dice el estado (por ejemplo, SUSPENDIDO). `evento_de_verificacion` guarda cada paso, con quién lo decidió, el fundamento y el mensaje.

Para comprobar. Si solo se guardara el estado, ¿qué pregunta de un profesor no podrías contestar?

CONCEPTO 25

Sin caché de permisos

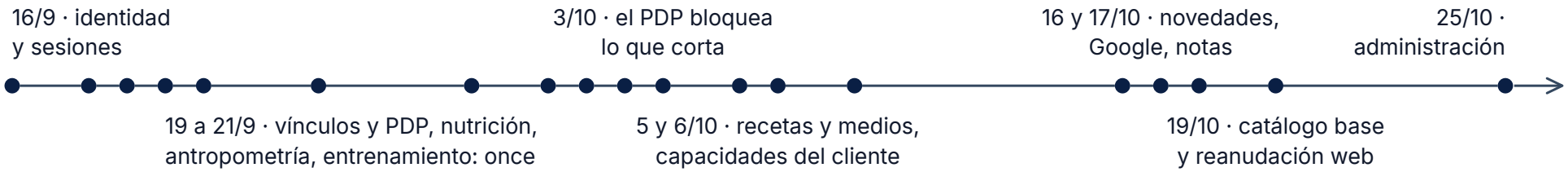
En criollo. El permiso se calcula en cada pedido con los datos de ese momento, en vez de recordarse.

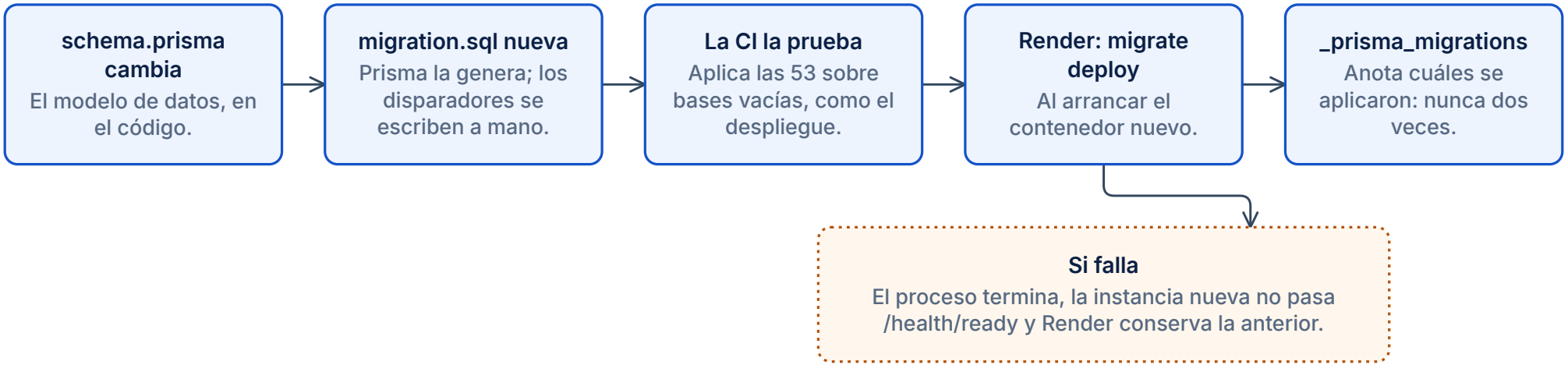
En BE. Por eso la suspensión vale desde el pedido siguiente, con la sesión del profesional abierta: el PDP vuelve a leer la verificación cada vez.

Para comprobar. ¿Qué ventaja y qué costo tiene no guardar los permisos en memoria?

¿Cómo cambia la base sin perder los datos que ya tiene?

Con migraciones: pasos numerados que se aplican una vez, en orden, y nunca se reescriben.





CONCEPTO 26

Migración

En criollo. Un archivo con los cambios de estructura de la base (crear una tabla, agregar una columna, un disparador), que se aplica una sola vez.
En BE. `20261025120100_administracion` creó cinco tablas (el rol administrativo, los códigos de incorporación y de recuperación, el segundo factor) y 17 disparadores. En test la aplicó la API sola, al arrancar.
Para comprobar. Una migración agrega una columna obligatoria a una tabla que ya tiene filas. ¿Qué tiene que decir la migración sobre esas filas?

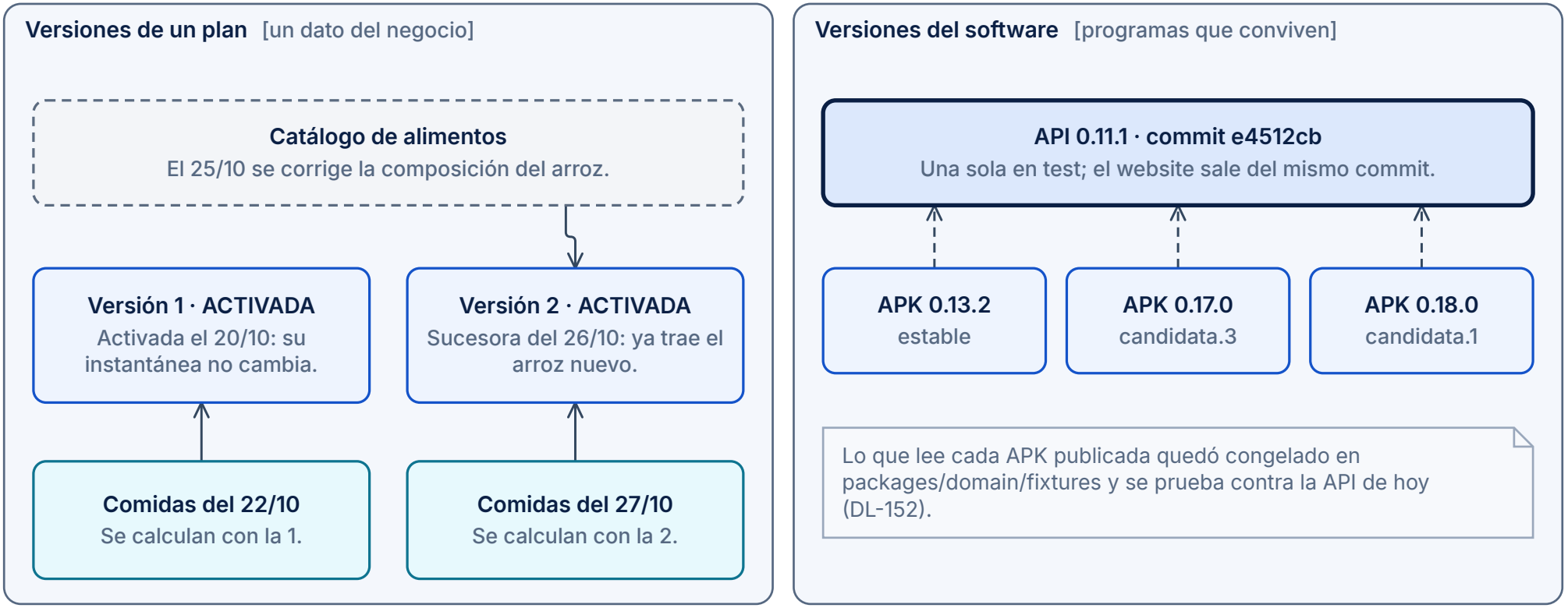
CONCEPTO 27

Lo aplicado no se edita

En criollo. Una vez aplicada en algún ambiente, una migración queda como está. Un error se corrige con otra nueva.
En BE. `_prisma_migrations` anota cada migración aplicada, con una suma de control del archivo. Editar una vieja deja dos historias: test conserva la que ya aplicó, y una base nueva tendría la editada.
Para comprobar. ¿Por qué editar una migración vieja «funciona en mi máquina» y rompe test?

¿Por qué la versión de un plan y la versión de la app son cosas distintas?

Una es un dato que se congela para conservar la historia; la otra es un programa que convive con los anteriores.



CONCEPTO 28

Compatibilidad hacia atrás

En criollo. Que lo nuevo siga funcionando con lo que ya está instalado y nadie actualizó.

En BE. Nadie actualiza una APK por la persona. Por eso la API no agrega un campo, un valor de enumeración o un `null` nuevo en lo que lee una APK publicada; y lo que una APK vieja no sabe mostrar se retiene (`X-BE-Capabilities`).

Para comprobar. ¿Por qué un campo nuevo en una respuesta puede romper una APK que no lo usa?

CONCEPTO 29

Contrato congelado

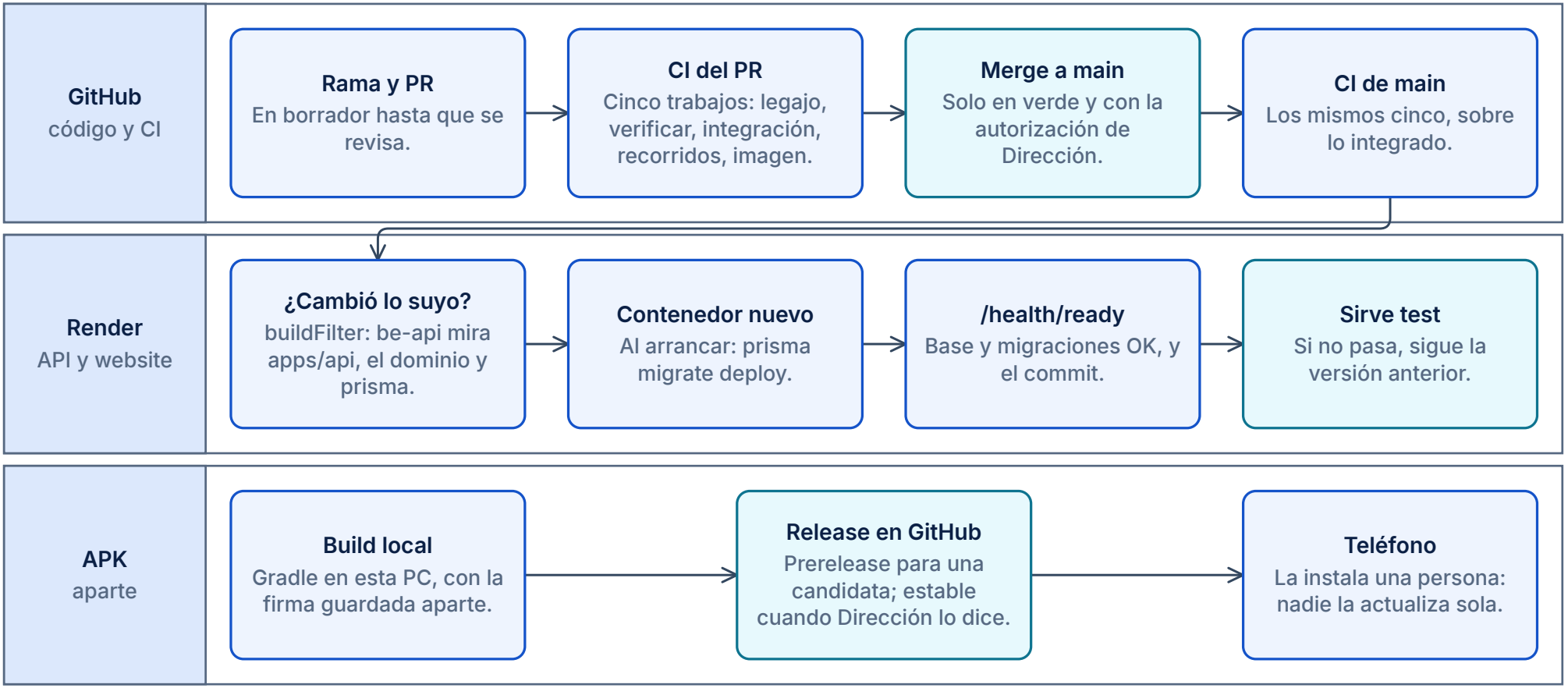
En criollo. Una copia fija de lo que espera una versión publicada, para probar que lo nuevo no la rompe.

En BE. `packages/domain/fixtures/lecturas-de-la-apk` guarda lo que leen la 0.13.2, la 0.17.0-candidata.3 y la 0.18.0-candidata.1; `lecturas-de-las-apk.int-spec.ts` se lo pide a la API de hoy.

Para comprobar. Si esa prueba falla, ¿se corrige el contrato congelado o la API? ¿Por qué?

¿Cómo llega un cambio a test, y una APK al teléfono?

La API y el website se despliegan solos cuando la CI de `main` termina bien. La APK se arma y se publica aparte.



CONCEPTO 30

Integración continua (CI)

En criollo. Cada cambio se prueba solo, en una máquina limpia, antes y después de juntarlo con el resto.
En BE. Cinco trabajos en `.github/workflows/ci.yml`. En `e4512cb`: 887 unitarias del dominio, 481 de scripts, 159 de la API, 909 de integración y 17 recorridos, todo en verde.
Para comprobar. ¿Por qué la CI corre otra vez en `main` si ya corrió en el PR?

CONCEPTO 31

Despliegue condicionado

En criollo. Publicar una versión solo si pasó sus controles, y poder quedarse con la anterior si algo falla al arrancar.
En BE. Render espera la CI (`autoDeployTrigger: checksPass`), mira si cambió algo de ese servicio (`buildFilter`) y pone la instancia nueva en servicio solo si `/health/ready` responde bien.
Para comprobar. Un PR cambia solo `apps/web`. ¿Qué servicios de Render se vuelven a desplegar?

¿Qué demuestra cada tipo de prueba, y qué no?

Cada prueba mira una parte. Ninguna sola alcanza, y ninguna automática dice si a una persona le sirve.

Tipo	Cuántas	Qué demuestra	Qué no demuestra
Unitarias del dominio	887	Reglas y cálculos puros: `evaluarAutorizacion`, `calcularNutrientes`.	La base, la red ni las pantallas.
De scripts y textos	481	Los textos de las pantallas (B05), el contraste, la compatibilidad de las APK.	El comportamiento con datos reales.
Unitarias de la API	159	Piezas de la API, aisladas.	Las transacciones reales.
Integración	909	La API con PostgreSQL real: disparadores, transacciones, PDP, carreras.	El navegador ni el teléfono.
Recorridos	17	Website, API y base juntos, en un navegador.	El teléfono ni la comprensión de una persona.
Imagen de la API	1 trabajo	Que el contenedor arranca, migra y responde «listo» con su commit.	Cómo se comporta con la carga de Render.
Escenarios en test	6 de 6	Que los datos de prueba cargados en test son los esperados (26/10).	Que alguien los entienda.
Prueba con personas	Parte A	Si se entiende sin ayuda. Hecha por Dirección el 27/10 la parte de Administración.	Lo que falta: partes B y C, y profesionales.

Los números son de la CI de `main` en el commit de referencia (`e4512cb`, 2026-10-26). Una prueba en verde dice que el código hace lo que la prueba pide; si la prueba pide poco, el verde dice poco.

CONCEPTO 32

Prueba de integración

En criollo. Una prueba que junta piezas reales, en vez de simularlas.

En BE. Cada archivo de `test/integration` corre la API contra un PostgreSQL 16 de verdad con las 53 migraciones: así se prueban los disparadores, los bloqueos y las transacciones.

Para comprobar. ¿Qué defecto de BE no podría encontrar nunca una prueba unitaria del dominio?

CONCEPTO 33

Prueba determinista

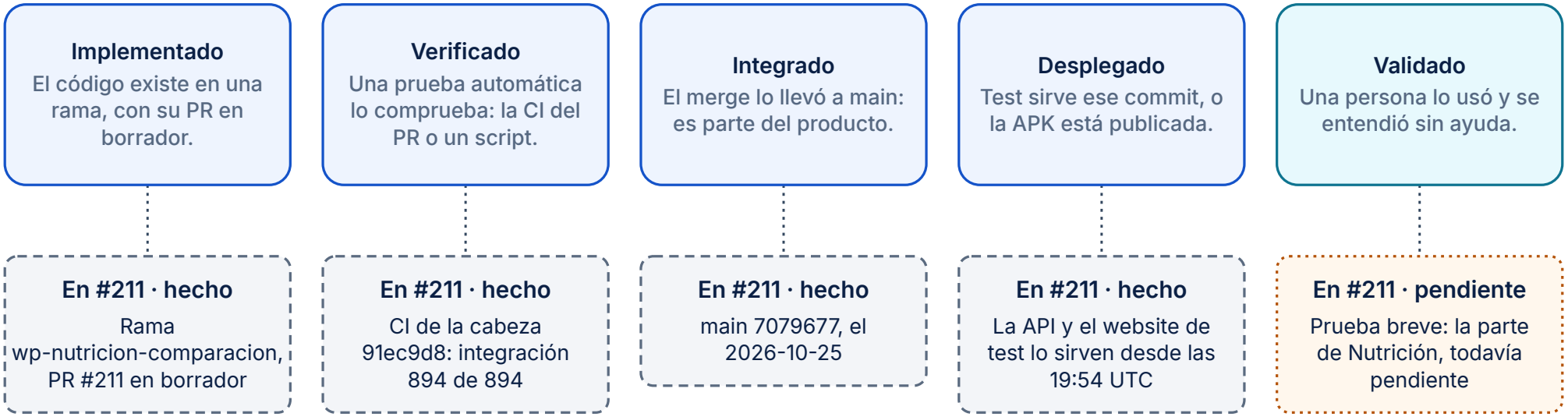
En criollo. Una prueba que da siempre el mismo resultado porque no depende de cuánto tarda algo.

En BE. En `main`, «el cruce» de `administracion.int-spec.ts` espera 800 ms para ver si la decisión quedó esperando. El PR #218, en borrador, la cambia por una barrera: sigue cuando PostgreSQL muestra el pedido bloqueado.

Para comprobar. Una prueba de carrera pasa en tu PC y falla una vez de cada veinte en la CI. ¿Qué te dice eso de la prueba?

¿Por qué «integrado», «desplegado» y «validado» no son lo mismo?

Un cambio tiene cinco dimensiones que se prueban por separado y se combinan: ninguna es un peldaño de otra.



Se combinan, no se suben como una escalera
Cada una responde su pregunta: «¿está hecho?», «¿lo comprueba una prueba?», «¿es parte del producto?», «¿se puede usar?» y «¿lo entiende una persona?». #218 está verificado y no integrado; una APK candidata se publica desde su propia rama.

CONCEPTO 34

Integración (merge)

En criollo. Llevar el trabajo de una rama a la rama principal. Desde ahí, es parte del producto.
En BE. Cada PR se integra con un merge desde GitHub, solo con la CI en verde y la rama al día con `main`.
Para comprobar. Si un PR está integrado pero la CI de `main` falló, ¿se despliega? ¿Por qué?

CONCEPTO 35

Validación humana

En criollo. Que una persona real use el sistema en una tarea real y se observe si entiende sin ayuda.
En BE. La prueba breve: las respuestas esperadas están en `OBSERVADOR.md`, y lo que pasó se anota sin corregir a quien prueba.
Para comprobar. ¿Por qué una prueba automática nunca reemplaza a una validación humana?

¿Por dónde seguir, y qué leer en el código?

La guía da el mapa. El cuaderno de datos baja a la base, y el plan de estudio lo convierte en práctica.

Los otros dos materiales

- **Cuaderno de datos** (`BE-cuaderno-de-datos.pdf`): qué es cada tabla, por qué hay tantas, transacciones, índices, disparadores y consultas SQL de lectura con datos inventados.
- **Plan de estudio** (`PLAN-DE-ESTUDIO.md`): cuatro sesiones de 20 a 30 minutos, con ejercicios. Las respuestas, y las de cada «Para comprobar» de esta guía, están aparte, en `RESPUESTAS.md`, para mirarlas después de intentar.

Leer el código en media hora, en este orden

1. `packages/domain/src/autorizacion.ts`: `evaluarAutorizacion`, las siete dimensiones.
2. `apps/api/src/nutricion/registro-de-comidas.service.ts`: `registrar`, el recorrido 1 entero.
3. `apps/api/src/plataforma/idempotencia.service.ts`: la transacción y la respuesta guardada.
4. `prisma/migrations/20260920100000_circuito_nutricional/migration.sql`: los disparadores de la historia.
5. `test/integration/administracion.int-spec.ts`: la suspensión, probada de punta a punta.

Qué no cubre esta guía

- Entrenamiento y Antropometría en detalle: siguen los mismos patrones (versiones, instantáneas, PDP), con sus propias tablas. El cuaderno las muestra por área.
- Por dentro de la APK y del website: navegación, estados de pantalla y estilos.
- El estado del proyecto y lo que falta para cerrarlo: está en `ESTADO-Y-RUTA-DE-CIERRE.md`, que se actualiza aparte.

Archivos revisados para escribirla

`apps/api/src` (sesion, http, plataforma, autorizacion, nutricion, analisis, administracion), `packages/domain/src` (autorizacion, contratos, administracion, compatibilidad de clientes), `prisma/schema.prisma` y las 53 migraciones, `test/integration` (administración, lecturas de las APK), `render.yaml`, `.github/workflows/ci.yml` y la corrida de la CI de `main` en `e4512cb`.

> **Cómo usarla para defender BE**
> Primero el nivel 1 (para qué), después un recorrido (cómo), y recién ahí las reglas. Cada afirmación, con su estado: lo implementado no es lo validado.